HelpDesk AI

A knowledge-base support assistant: answers from a team's own documents, cites the exact passage, refuses when unsure, and shows what customers ask.

Kriti Singhal · Technical documentation · October 2026

GitHub: github.com/krit566/ai-support-assistant

Contents

1. Overview

2. Who uses it and how

3. Architecture

4. Data model

5. How a question is answered

6. Engineering decisions

7. Evaluation

8. Analytics and the feedback loop

9. Documents and the public widget

10. Security and privacy

11. Testing and delivery

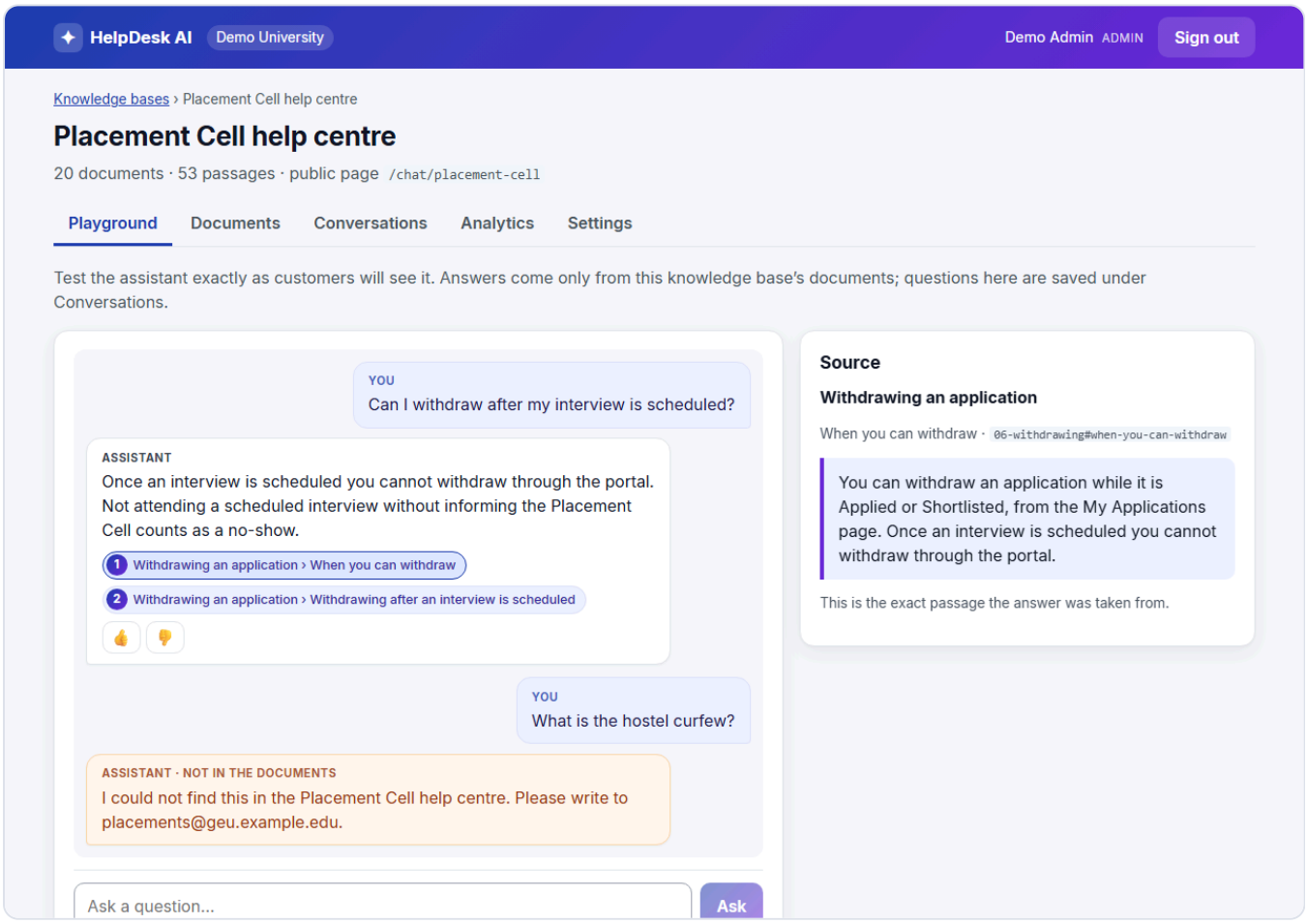
12. Limitations and next steps

13. Interview questions

1 Overview

HelpDesk AI is a support assistant that any team can set up on its own documents. A team uploads help articles, policies or FAQs (PDF, Markdown or text). Customers then ask questions on a public chat page; the assistant answers **only from those documents**, shows **the exact passage each answer came from**, and says **"I could not find this"** instead of guessing when the documents do not cover a question. The team sees what customers ask, what was refused, and which answers were rated unhelpful, which tells them what to write next.

96% right article in the top 3 (held-out test questions)	**9** database tables, schema managed by Alembic migrations	**53** automated tests: 42 backend on real PostgreSQL, 11 frontend	**0** axe accessibility violations in the tested pages

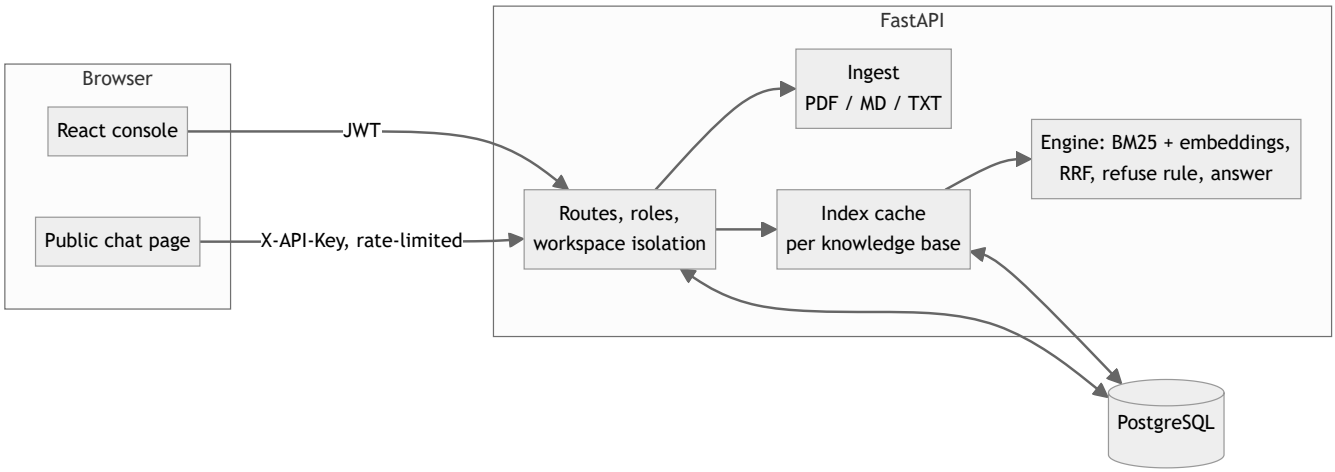


React + TypeScript · FastAPI · PostgreSQL · SQLAlchemy and Alembic · hand-written BM25 with embeddings (retrieval-augmented generation) · JWT and bcrypt · Docker · GitHub Actions.

2 Who uses it and how

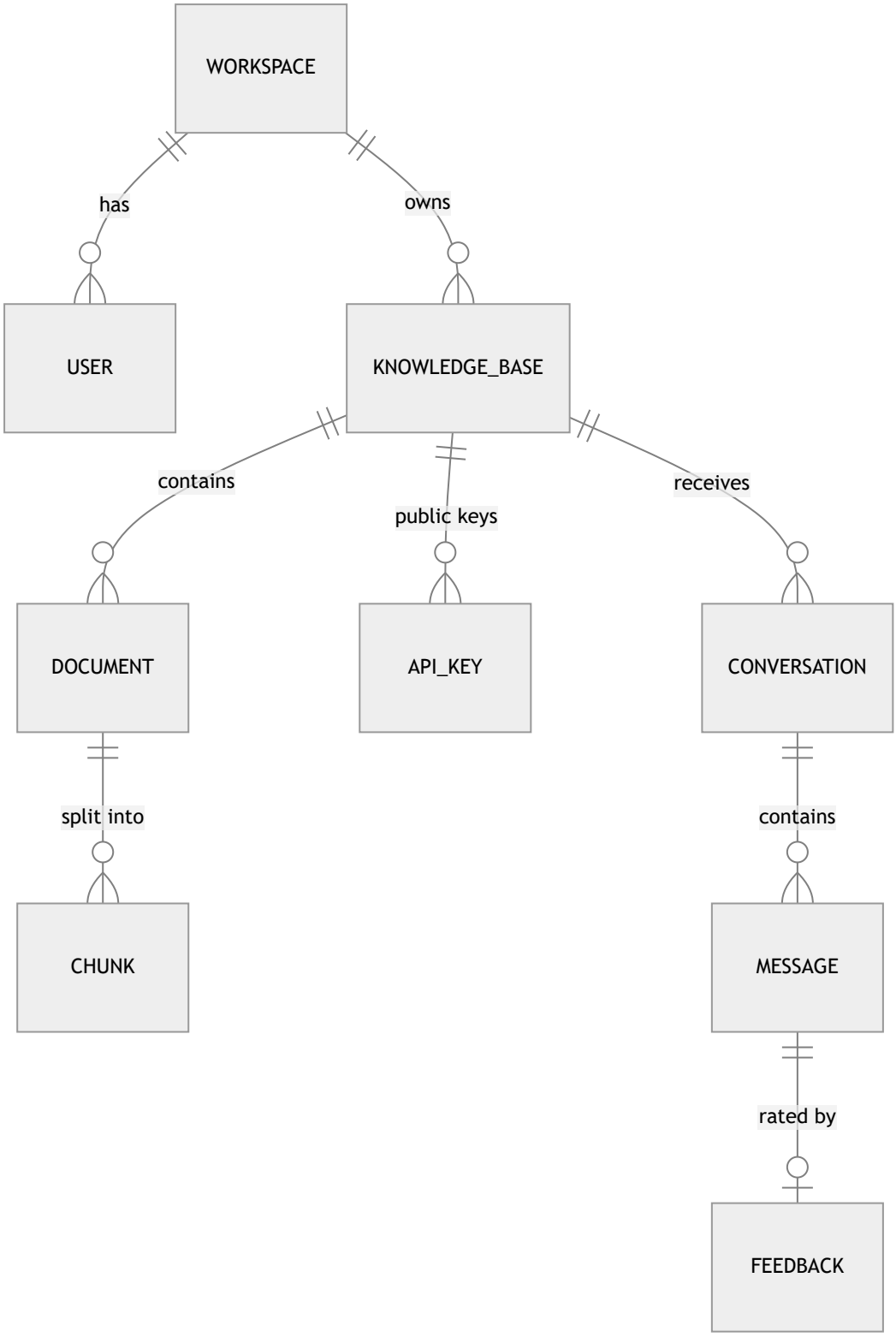
Person	What they do
Admin	Creates the workspace and knowledge bases, uploads documents (drag and drop), checks how each was split into passages, re-uploads new versions, creates public keys, invites agents.
Agent	Tests questions in the playground, clicks citations to read the source passage, rates answers, reads conversations and analytics.
Customer	Asks on the public chat page `/chat/<knowledge base>?key=…` without signing in.
Support lead	Uses analytics: volume per day, refusal rate, helpful rate, answer time, **gaps** (refused questions grouped by topic) and low-rated answers with comments.

3 Architecture



Part	Responsibility
`backend/assistant/`	The retrieval engine: chunking, tokenising, BM25 written out by hand, LSA / sentence-transformers / API embeddings, Reciprocal Rank Fusion, the refuse rule, extractive and LLM answers.
`backend/app/ingest.py`	Upload checks (size, type, content) and parsing: Markdown by "##" sections, PDFs by page, text by paragraphs.
`backend/app/index.py`	One in-memory index per knowledge base, rebuilt when its `index_version` changes; threshold scaling; attaches the cited passage text to each answer.
`backend/app/routes/`	Auth, knowledge bases and documents, chat and analytics, public widget.
`backend/app/analytics.py`	Per-day volume, refusal rate, percentiles (SQL `percentile_cont`), gaps, low-rated answers.
`frontend/src/`	React Router pages, one shared chat component for the playground and the public page, hand-written SVG charts.

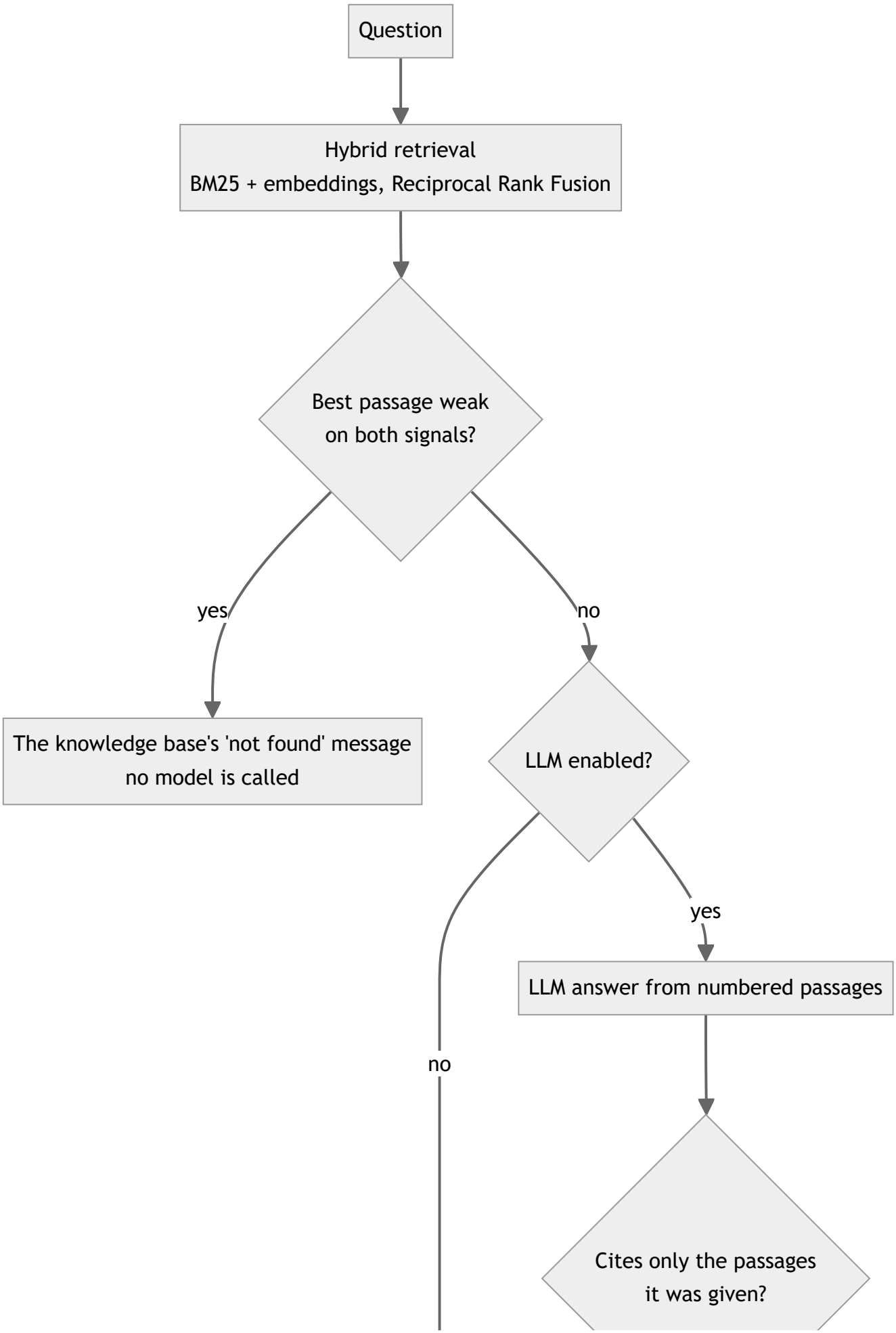
4 Data model

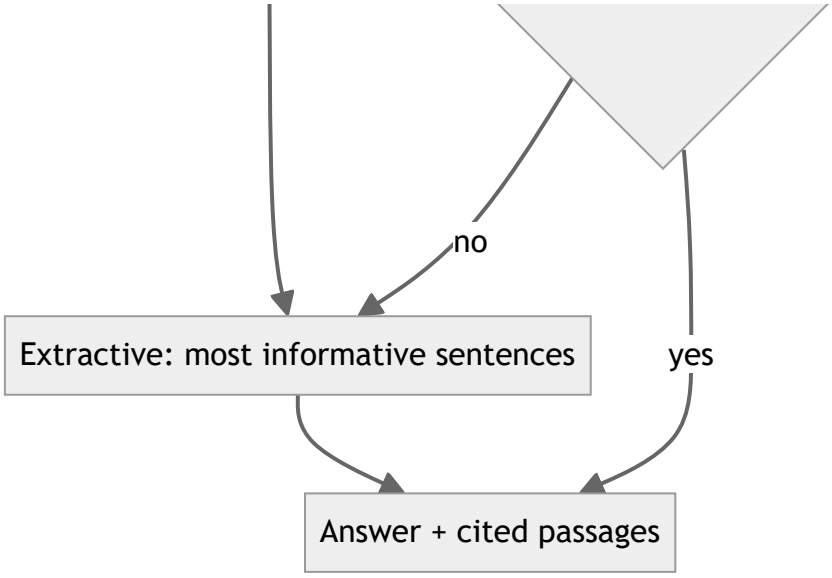


- **Documents** keep their extracted text, so they can be re-chunked later without the original file; a re-upload with the same name increments `version`.
- **Messages** store the cited passages (JSONB) as they were at answer time, so an old answer still shows its evidence after the document changes.

- **Chunks** can store an embedding vector (`float[]`) for embedders that do not depend on the rest of the corpus; LSA vectors are recomputed at rebuild.
- All foreign keys use `ON DELETE CASCADE`; deleting a knowledge base removes its documents, keys and conversations.

5 How a question is answered


Question
Hybrid retrieval
BM25 + embeddings, Reciprocal Rank Fusion
Best passage weak
on both signals?
yes
no
The knowledge base's 'not found' message
no model is called
LLM enabled?
yes
LLM answer from numbered passages
no
Cites only the passages
it was given?



1. **BM25** scores exact words; rare words count more (IDF), repeats saturate (`k1`), long passages are not favoured (`b`).
2. **Embeddings** score meaning; LSA (TF-IDF + truncated SVD) offline by default, plain TF-IDF for knowledge bases under 8 passages.
3. **Reciprocal Rank Fusion** merges the two rankings by position, so their different score scales never need to be compared.
4. **Refuse rule:** if the best passage is weak on both signals the assistant refuses with the knowledge base's own message.
5. **Answer:** extractive by default; with an LLM configured, the reply must cite passage numbers it was given, or the extractive answer is used.

6 Engineering decisions

Index in memory, version in the database

Every document change increments `knowledge_bases.index_version` in the same transaction (as an SQL expression, so two concurrent uploads both count). Before answering, a process compares its cached version with the database's and rebuilds if they differ, which keeps several server processes consistent without a message bus.

Why not pgvector? BM25 needs word statistics over the whole knowledge base and LSA is fitted on all passages, so both are rebuilt from all passages anyway; for a few thousand passages that takes well under a second. pgvector would move nearest-neighbour search into the database for very large collections, but keyword search would still need a separate full-text index and the rankings would still be fused in the application.

Thresholds do not transfer between corpus sizes

The refuse thresholds were tuned on the 53-passage placement-cell corpus. The first product version refused **every** question on a small knowledge base: BM25 scores shrink with corpus size (a word found in 1 of 2 passages weighs 0.69, in 1 of 53 it weighs 3.58), so the tuned bar of 3.5 was unreachable. The BM25

threshold is now expressed in units of "one rare word" and converted for each knowledge base. On the placement corpus the conversion is exactly a no-op.

> A test uploads the 20 evaluated articles through the API and checks that the product makes the **same rankings and the same refuse decisions** as the evaluated engine for all 46 evaluation questions, so the evaluation numbers describe the product.

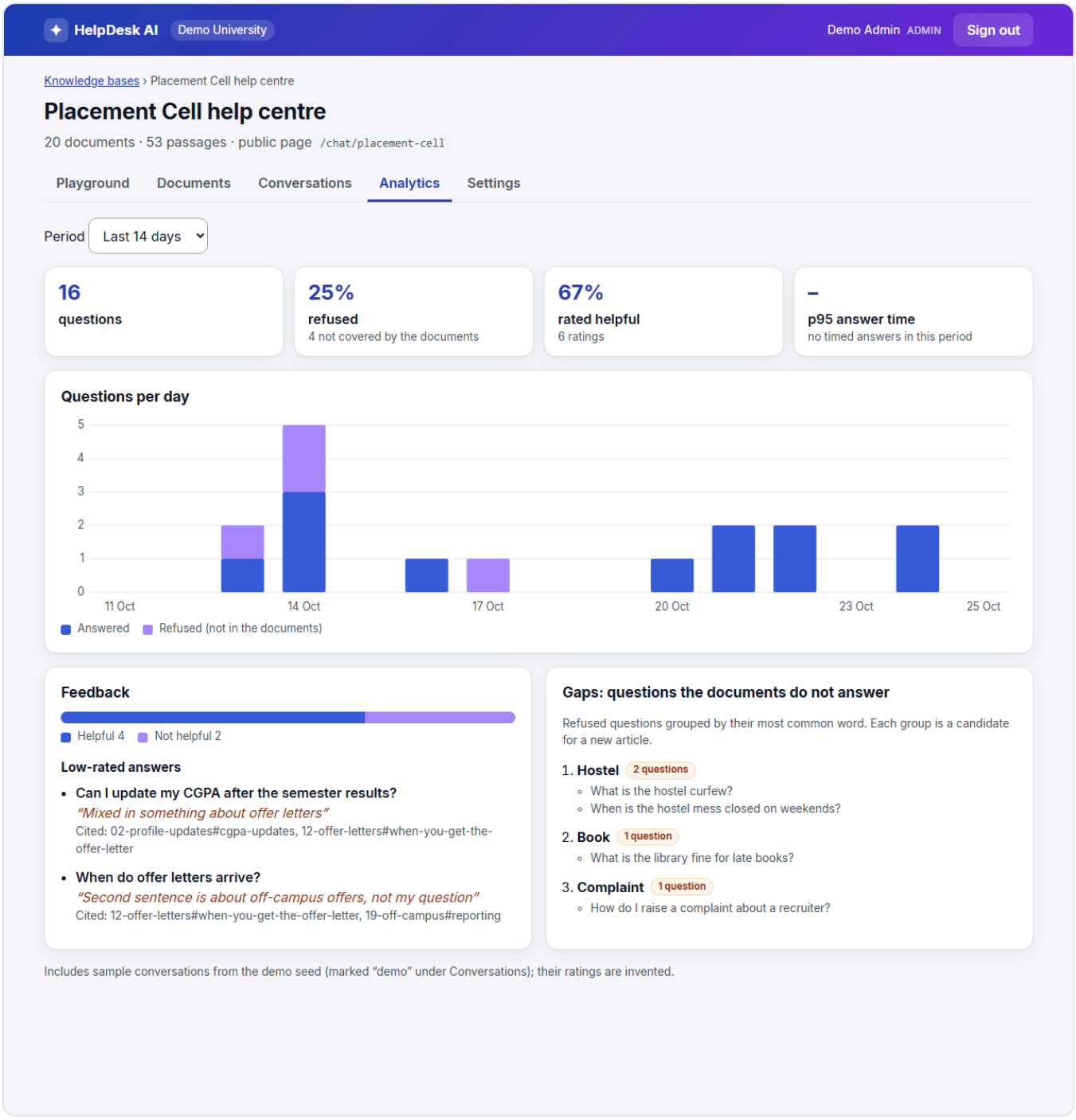
7 Evaluation

46 questions written for this project: 37 answerable (each labelled with its article and one key fact) and 9 not covered. Dev split (14) for choosing thresholds, test split (32) for reporting; offline LSA embedder, extractive answers.

Metric (test split)	Value
Right article ranked first (recall@1), hybrid	89% (24/27)
Right article in the top 3 (recall@3)	96% (26/27)
MRR	0.92
Answer correct: right article and the key fact	70% (19/27)
Unanswerable questions correctly refused	3/5
Answerable questions wrongly refused	1/27

> Refusing is the weak spot. In the demo, "Which buses go to the city campus?" is answered with the off-campus offers article because "campus" matches, and "How do I raise a complaint about a recruiter?" is refused although a grievances article exists. The articles and questions were written by one person and the test set is small (one question ≈ 3 points), so all scores are optimistic. CI fails the build if recall@3 drops below 0.90.

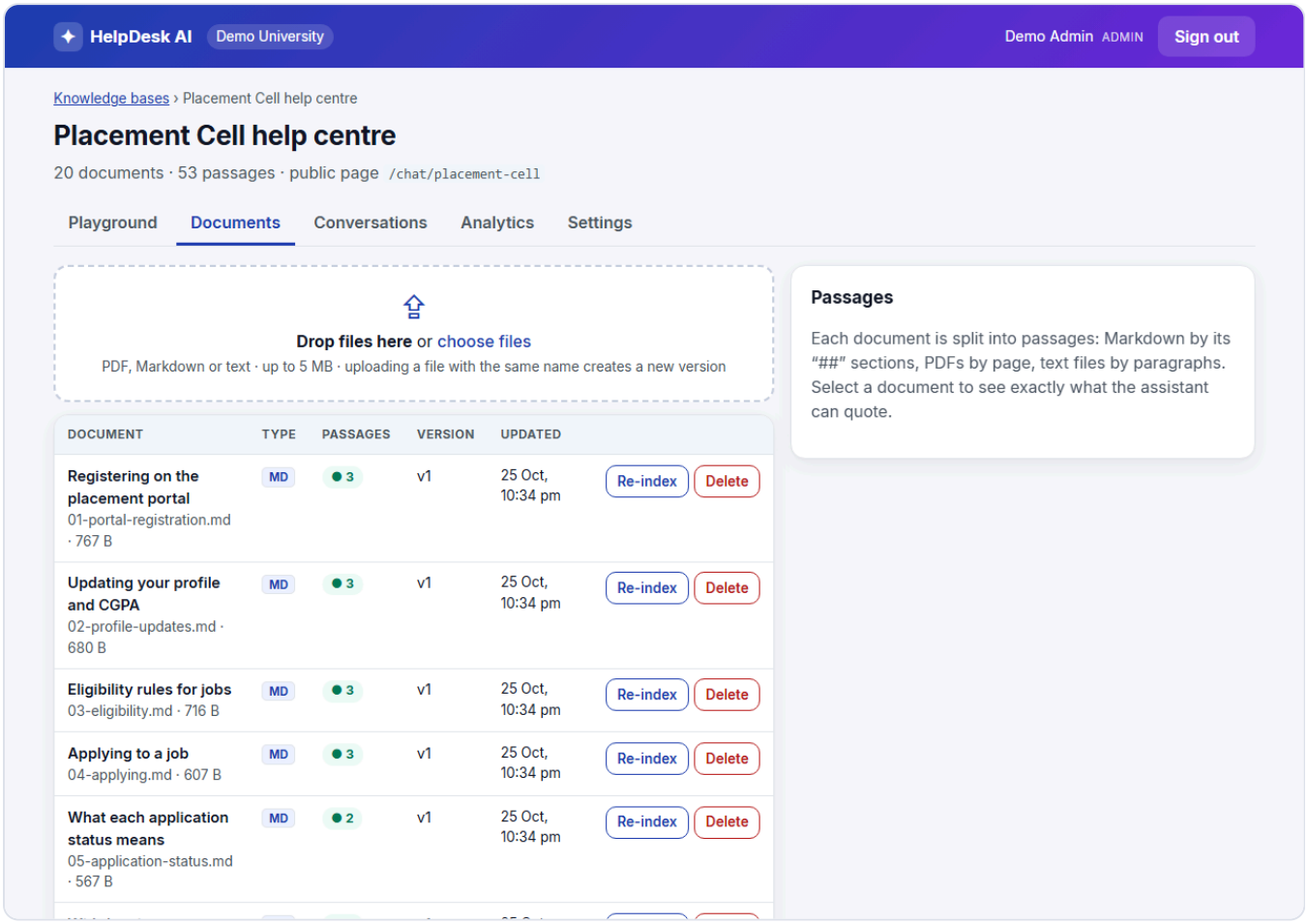
8 Analytics and the feedback loop



- **Questions per day**, answered vs refused, every day in the range including empty days.
- **Refusal rate, helpful rate, p50/p95 answer time** (SQL `percentile_cont`; shown as "–" when nothing was timed).
- **Gaps:** refused questions grouped by their most frequent informative word; each group is a candidate article.
- **Low-rated answers** with the customer's question, comment and the passages that were cited.

> The demo seed asks 16 sample questions through the real engine; their thumbs up/down are invented and the conversations are labelled "demo" in the UI.

9 Documents and the public widget



Upload problem	Response
Over the size limit (5 MB by default)	413
Not PDF / Markdown / text, a ".pdf" that is not a PDF, non-UTF-8 or binary text	415
Empty, blank, unreadable PDF or no text layer	422
Same file again	200, nothing changes
Same name, new content	200, version + 1, old passages replaced, index refreshed

The public page works with a **publishable key** created per knowledge base. The key is visible to visitors, so it can only ask questions of that one knowledge base, is stored only as a SHA-256 hash, can be revoked, and is rate-limited per key with a sliding 60-second window (429 with `Retry-After`).

10 Security and privacy

Concern	How
Passwords	bcrypt; same error and similar timing for unknown email and wrong password.
Sessions	JWT, HS256 pinned, expiry; the role is re-read from the database on every request.
Isolation	All queries scoped to the caller's workspace; other workspaces' ids return 404 (tested for 9 endpoints).
Roles	Agents ask, rate and read; only admins change documents, keys, users and settings.
Uploads	Size limit, extension allow-list plus content check, file names sanitised (no paths).
LLM	Optional; key only on the server; citations validated.
Logs	JSON lines with request ID, outcome, sources and timing; never the question text.

11 Testing and delivery

Suite	Covers
Engine (15)	Chunking, BM25, retrieval on known questions, refusals, LLM citation check.
Product API (21)	Accounts, uploads and 8 kinds of bad upload, versions, deletion, refusal message, product index equals evaluated engine, isolation, roles, feedback, analytics, keys, rate limit, revocation.
Units (6)	Threshold scaling, rate-limit window with a fake clock, file-name sanitising, chunking.
Frontend (11)	Sign-in, redirects, citation panel, follow-ups, refusals, shareable links, uploads, analytics, public key handling, axe scans.

Backend tests run on a **real PostgreSQL** (an embedded server locally, a postgres service container in CI) with the schema built by running the Alembic migrations. GitHub Actions runs backend tests and the evaluation gate, frontend tests and build, then builds both Docker images. `docker-compose.yml` runs PostgreSQL, the API and nginx (serving the app and forwarding `/api`); `render.yaml` deploys the same pieces to Render.

```
docker compose up --build                          # http://localhost:8080
docker compose exec backend python -m app.seed     # demo workspace
cd backend && python -m pytest -q                  # 42 tests
cd frontend && npm test                            # 11 tests
```

12 Limitations and next steps

- Refusal and sentence choice are the weak parts; next, evaluate pretrained embeddings and the LLM mode on the same split, and tune thresholds per knowledge base from thumbs-down data.
- Rate limiter and index cache are per process; with several instances the limiter moves to Redis.
- Uploads are parsed inside the request; large PDFs need a background job queue. No OCR for scanned PDFs.

- The session token is kept in `localStorage`; an httpOnly cookie with CSRF protection is stronger for production.

13 Interview questions

Why fuse rankings with RRF instead of adding scores?

BM25 scores are unbounded and cosine similarities lie in [−1, 1]; adding them lets one dominate. RRF uses only positions.

Why did the thresholds need rescaling?

BM25's IDF depends on how many passages exist, so the same "one rare word" match scores 0.69 in a 2-passage knowledge base and 3.58 in a 53-passage one. A fixed threshold either refuses everything on small bases or nothing on large ones.

How does the cache stay correct with several server processes?

Each change bumps index_version in the same transaction; every process compares its cached version with the database before answering.

Why return 404 for another workspace's ids?

403 would confirm the id exists; 404 reveals nothing about other teams' data.

Why bcrypt for passwords but SHA-256 for API keys?

Passwords are short and guessable, so hashing must be slow; API keys are 24 random bytes, so a fast hash is already infeasible to reverse.

What still protects a knowledge base if its public key leaks?

The key can only ask questions of that one knowledge base, is rate-limited, and can be revoked without affecting other keys.